

PostgreSQL Performance Analysis and Optimization Report

1. Issues Identified

Під час аналізу було виявлено декілька проблем продуктивності:

- повільні аналітичні запити з агрегацією даних;
- неефективний JOIN між таблицями customers та orders;
- пошук по email без відповідного індексу;
- повне сканування таблиці orders під час пошуку за статусом;
- блокування транзакцій під час одночасного оновлення записів у таблиці customers;
- використання широкої таблиці customer_events_wide з великою кількістю атрибутів.

2. How the Issues Were Detected

Для аналізу використовувалися такі інструменти PostgreSQL:

- **pg_stat_statements** для пошуку запитів, які створювали найбільше навантаження;
- **EXPLAIN ANALYZE** для перегляду планів виконання запитів та вимірювання часу їх виконання;
- **pg_stat_activity** для аналізу активних сесій і довгих транзакцій;
- **pg_locks** для виявлення блокувань між транзакціями.

3. Problematic SQL Queries

Найбільше навантаження створювали:

1. Агрегаційний запит по таблиці customer_events_wide.
2. JOIN таблиць customers та orders з використанням COUNT() і SUM().
3. Пошук замовлень зі статусом paid.
4. Пошук email через умову LIKE '%gmail%'.
5. UPDATE-запити до таблиці customers, які викликали блокування.

4. Implemented Changes

Було реалізовано такі оптимізації:

Orders Query

Створено індекс по полю status таблиці orders.

Email Search

Створено GIN Trigram Index для поля email.

Heavy JOIN

Замість повторного виконання агрегації для таблиці orders було створено Materialized View із попередньо підрахованими даними.

Aggregation Query

Було створено composite index по полях:

- event_time
- customer_id
- event_type

Також було проаналізовано можливість використання Materialized View та партиціювання таблиці.

Database Maintenance

Виконано VACUUM ANALYZE для основних таблиць бази даних.

Concurrency Analysis

Було виявлено блокування між транзакціями, які одночасно змінювали один і той самий запис у таблиці customers. Причиною були довгі транзакції та конкурентні UPDATE-запити.

5. Results After Optimization

Orders Query

Час виконання зменшився:

- до оптимізації: 12.9 ms
- після оптимізації: 5.1 ms

Покращення приблизно на 60%.

Email Search

Час виконання зменшився:

- до оптимізації: 1.71 ms
- після оптимізації: 0.05 ms

Покращення приблизно на 97%.

Heavy JOIN

Час виконання зменшився:

- до оптимізації: 32.5 ms
- після оптимізації: 8.1 ms

Покращення приблизно на 75%.

Aggregation Query

Composite index не дав значного ефекту, оскільки запит обробляв майже половину таблиці. Було зроблено висновок, що для такого типу навантаження більш ефективними є Materialized Views або партиціювання.

6. Before and After Comparison

Query	Before	After	Result
Orders Search	12.9 ms	5.1 ms	Faster
Email Search	1.71 ms	0.05 ms	Much Faster
Heavy JOIN	32.5 ms	8.1 ms	Faster
Aggregation Query	172 ms	176 ms	No significant improvement

Conclusion

Було проаналізовано продуктивність PostgreSQL під навантаженням та виявлено декілька типових проблем: повільні агрегації, неефективні JOIN, відсутність індексів і блокування транзакцій.

Для оптимізації були використані індекси, Materialized View, VACUUM ANALYZE та аналіз блокувань. Найкращі результати дали GIN-індекс для пошуку email та використання Materialized View для складного JOIN-запиту. Також було встановлено, що не всі проблеми можуть бути вирішені індексами, а для великих агрегацій доцільно використовувати попередню агрегацію даних або партиціювання таблиць.